



GESTIÓN DE CALIDAD Y PRUEBAS

INFORME DE PRUEBAS

FILMATE

Validación técnica del frontend de usuario: pruebas unitarias, integración, E2E, accesibilidad, build, seguridad de dependencias y preparación para SonarQube.

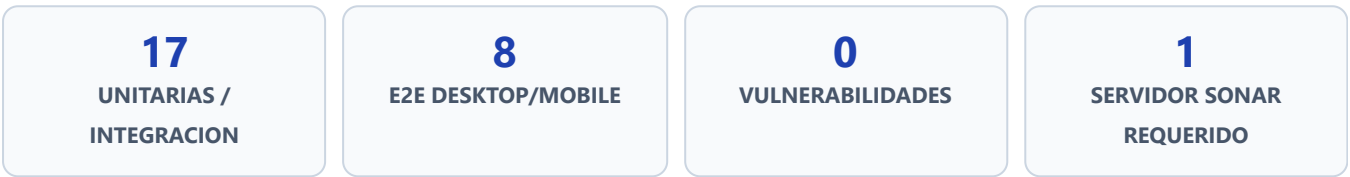
Campo	Información
Código del documento	IP-FILMATE-USR-001
Proyecto	FILMATE UserFrontend
Versión	1.0
Fecha de ejecución	23 de junio de 2026
Entorno	Windows 11, Node.js v24.11.1, npm 11.6.2
Estado	VALIDADO
Responsable	Equipo del proyecto FILMATE

1. Resumen ejecutivo

Se implementó una base completa de pruebas automatizadas para el frontend de usuario. La verificación integral final ejecutó lint, cobertura, build y pruebas E2E con backend mock local. El resultado fue satisfactorio.

Resultado final: npm run test:all terminó correctamente. Se validaron 17 pruebas unitarias/integración y 8 pruebas E2E en escritorio y móvil.

Indicadores principales



Contenido del informe

1. Resumen ejecutivo

2. Matriz de resultados

3. Cobertura generada

4. Antes, durante y despues

5. Tipos de pruebas cubiertos

6. SonarQube

7. Archivos de evidencia
8. Recomendaciones

9. Inventario completo de pruebas

10. Scripts y dependencias de calidad

11. Configuracion SonarQube

12. Inventario de archivos creados/modificados

13. Anexos documentales completos

2. Matriz de resultados

Comando	Tipo	Resultado	Observación
npm run lint	Estático	OK	0 errores y 0 warnings.
npm run test:coverage	Unitarias / integración	OK	4 archivos de prueba, 17 tests pasaron. Generó coverage/lcov.info.
npm run build	Producción	OK	Build generado en dist/. Advertencia no bloqueante por chunk mayor a 500 kB.
npm run test:e2e	E2E / accesibilidad	OK	8 tests pasaron en chromium y mobile-chrome.
npm audit --json	Seguridad	OK	0 vulnerabilidades.
npm run sonar	SonarQube	CONDICIONADO	Scanner listo; requiere servidor SonarQube accesible y token.

3. Cobertura generada

Métrica	Resultado	Comentario
Statements	8.54%	Cobertura inicial midiendo todo src.
Branches	12.65%	Quedan pantallas grandes sin unitarias específicas.
Functions	8.47%	Los flujos principales están reforzados con E2E.
Lines	8.94%	Reporte disponible para SonarQube en coverage/lcov.info.

4. Antes, durante y después

Antes

- No existía script de pruebas.
- No había cobertura ni configuración SonarQube.
- No había E2E automatizado.
- npm audit reportaba 3 vulnerabilidades corregibles.

Durante

- Se instalaron Vitest, Testing Library, Playwright, Axe y sonar-scanner.
- Se agregó runner E2E propio para Windows.
- Se corrigieron hallazgos de lint y accesibilidad.
- Se mejoró normalización de datos anidados en filmateApi.

Después

- La suite integral terminó en estado OK.
- Las dependencias quedaron sin vulnerabilidades conocidas por npm audit.
- La evidencia y los comandos quedaron documentados en docs/testing.
- SonarQube quedó preparado para ejecutarse cuando exista servidor y token.

5. Tipos de pruebas cubiertos

Tipo	Herramienta	Objetivo
Lint / estático	ESLint	Detectar errores, imports sin uso y reglas de hooks.
Unitarias	Vitest	Validar sesión y normalizadores de datos.
Integración UI	Testing Library	Validar login, redirección y rutas protegidas.
E2E desktop/móvil	Playwright	Validar navegación, catálogo, filtros y login con API mock.
Accesibilidad	Axe + Playwright	Detectar violaciones automatizadas en la pantalla de login.
Seguridad dependencias	npm audit	Verificar vulnerabilidades conocidas.
Calidad continua	SonarQube	Consumir cobertura LCOV y reglas de calidad.

6. SonarQube

Se agregó **sonar-project.properties** y se configuró el reporte de cobertura JavaScript con **coverage/lcov.info**. El intento local contra **http://127.0.0.1:9000** no pudo completarse porque no había servidor SonarQube escuchando.

Comando recomendado: `npm run sonar -- --define sonar.host.url=<URL> --define sonar.token=<TOKEN>`

7. Archivos de evidencia

Archivo	Propósito
docs/testing/PLAN_PRUEBAS.md	Estrategia, alcance y criterios de aceptación.
docs/testing/BITACORA_PRUEBAS.md	Registro antes, durante y después con resultados.
docs/testing/SONARQUBE.md	Guía de ejecución SonarQube.
coverage/lcov.info	Reporte de cobertura para SonarQube.
playwright-report/index.html	Reporte navegable de pruebas E2E.
docs/testing/Informe_Pruebas_FILMATE.pdf	Informe portable para entrega.

8. Recomendaciones

1. Agregar pruebas unitarias específicas para DetallePelicula, Dulceria y Social.
2. Configurar SonarQube en CI con SONAR_HOST_URL y SONAR_TOKEN.
3. Evaluar code splitting para reducir el chunk grande asociado a librerías de PDF/captura.
4. Mantener npm audit como parte de la validación previa a entrega.

FILMATE UserFrontend - Informe generado automáticamente desde scripts/testing/generate-testing-report.mjs

9. Inventario completo de pruebas automatizadas

9.1 Pruebas unitarias e integracion

Archivo	Casos	Cobertura funcional
authSession.test.js	4	Sesion invitado, sesion registrada, limpieza y JSON invalido.
filmateApi.test.js	8	Normalizadores, busqueda, login, errores traducidos y funciones.
ProtectedRoute.test.jsx	2	Redireccion de invitados y acceso registrado.
IniciarSesion.test.jsx	3	Validacion, login registrado y acceso invitado.

9.2 Pruebas E2E y accesibilidad

Archivo	Caso	Proyecto Playwright
auth-and-navigation.spec.js	guest can enter the public catalog and does not see social navigation	chromium + mobile-chrome
auth-and-navigation.spec.js	registered login opens the catalog and enables social navigation	chromium + mobile-chrome
catalog-and-accessibility.spec.js	catalog renders mocked movies and filters by genre	chromium + mobile-chrome
catalog-and-accessibility.spec.js	login page has no critical automated accessibility violations	chromium + mobile-chrome

Cada caso E2E se ejecuta contra backend mock local y se repite en escritorio y movil.

10. Scripts y dependencias de calidad

10.1 Scripts npm disponibles

Script	Comando real
npm run dev	vite
npm run build	vite build
npm run lint	eslint .
npm run preview	vite preview
npm run test	vitest run
npm run test:watch	vitest
npm run test:coverage	vitest run --coverage
npm run test:e2e	node scripts/e2e/run-playwright.mjs
npm run test:e2e:headed	node scripts/e2e/run-playwright.mjs --headed
npm run test:all	npm run lint && npm run test:coverage && npm run build && npm run test:e2e
npm run report:testing	node scripts/testing/generate-testing-report.mjs
npm run sonar	sonar-scanner

10.2 Dependencias agregadas para pruebas y analisis

Dependencia	Version declarada
@axe-core/playwright	^4.12.1
@playwright/test	^1.61.1
@testing-library/jest-dom	^6.9.1
@testing-library/react	^16.3.2
@testing-library/user-event	^14.6.1
@vitest/coverage-v8	^4.1.9
jsdom	^29.1.1
sonar-scanner	^3.1.0
vitest	^4.1.9

11. Configuración SonarQube incluida

El proyecto ya contiene un archivo de propiedades listo para que SonarQube consuma fuentes, pruebas y cobertura LCOV.

```
sonar.projectKey=filmate-userfrontend
sonar.projectName=Filmate UserFrontend
sonar.sourceEncoding=UTF-8

sonar.sources=src
sonar.tests=src,tests
sonar.test.inclusions=src/**/*.test.js,src/**/*.test.jsx,tests/**/*.spec.js
sonar.exclusions=dist/**,coverage/**,node_modules/**,playwright-report/**,test-results/**,docs/manual-
usuario/capturas/**,src/**/*.test.js,src/**/*.test.jsx
sonar.javascript.lcov.reportPaths=coverage/lcov.info

# Configure these outside the repository when running against SonarQube/SonarCloud:
# SONAR_HOST_URL=http://localhost:9000
# SONAR_TOKEN=<token>
```


12. Inventario de archivos creados o modificados

Grupo	Archivos
Configuracion	package.json, package-lock.json, vite.config.js, eslint.config.js, playwright.config.js, sonar-project.properties
Runner E2E	scripts/e2e/run-playwright.mjs
Reporte PDF	scripts/testing/generate-testing-report.mjs, docs/testing/Informe_Pruebas_FILMATE.pdf
Pruebas unitarias/integracion	src/Component/*.test.js, src/Component/*.test.jsx, src/test/setup.js
Pruebas E2E	tests/e2e/*.spec.js
Documentacion	docs/testing/PLAN_PRUEBAS.md, BITACORA_PRUEBAS.md, SONARQUBE.md
Mejoras de app	Header.jsx, IniciarSesion.jsx, MenuPrincipal.jsx, filmateApi.js y limpieza de imports React sin uso

Anexo A. Plan de pruebas completo

Plan de pruebas - Filmate UserFrontend

Fecha de referencia: 2026-06-23.

Objetivo

Validar el frontend de usuario de Filmate con una combinacion de pruebas automaticas y analisis estatico que cubra:

- Calidad estatica del codigo.
- Funciones puras y normalizacion de datos.
- Integracion de componentes con rutas, sesion y API mockeada.
- Flujos E2E en escritorio y movil.
- Accesibilidad automatizada basica.
- Build de produccion.
- Auditoria de dependencias.
- Preparacion de cobertura para SonarQube.

Alcance

Se modifica solo el proyecto `FILMATE_UserFrontend`.

Quedan dentro del alcance:

- `src/Component/filmateApi.js`
- `src/Component/authSession.js`
- rutas protegidas
- login y navegacion
- cartelera con filtros
- configuracion de pruebas y reportes
- documentacion en `docs/testing`

Quedan fuera del alcance:

- Backend real.
- Repositorios externos.
- Analisis SonarQube contra servidor real cuando no exista `sonar.host.url` accesible ni token.

Tipos de prueba

Tipo	Herramienta	Comando	Evidencia
Lint / estatico	ESLint	<code>npm run lint</code>	salida limpia
Unitarias	Vitest	<code>npm run test</code>	tests de normalizadores y sesion
Integracion de componentes	Vitest + Testing Library	<code>npm run test:coverage</code>	login, redireccion y ruta protegida
Cobertura	V8 / Vitest	<code>npm run test:coverage</code>	<code>coverage/lcov.info</code>
E2E desktop/mobile	Playwright	<code>npm run test:e2e</code>	<code>playwright-report/index.html</code>
Accesibilidad automatizada	Axe + Playwright	<code>npm run test:e2e</code>	prueba sin violaciones criticas/moderadas en login
Build	Vite	<code>npm run build</code>	<code>dist/</code>
Seguridad dependencias	npm audit	<code>npm audit --json</code>	0 vulnerabilidades
SonarQube	sonar-scanner	<code>npm run sonar -- --define sonar.host.url=<url></code>	requiere servidor y token

Criterios de aceptacion

- `npm run lint` debe finalizar en codigo 0.
- `npm run test:coverage` debe generar `coverage/lcov.info`.
- `npm run test:e2e` debe finalizar en codigo 0 en los proyectos `chromium` y `mobile-chrome`.
- `npm run build` debe finalizar en codigo 0.
- `npm audit --json` debe reportar `total: 0`.
- SonarQube debe poder ejecutarse cuando exista servidor accesible y token valido.

Datos de prueba

Los E2E usan `scripts/manual/mock-server.mjs` como backend controlado. El runner de E2E levanta:

- Mock API: `http://127.0.0.1:8011`
- Vite app: `http://127.0.0.1:5173`

El puerto del backend mock se separo de `8000` para no interferir con un backend local real.

Brechas conocidas

- La cobertura global unitaria es baja porque se empezo a medir todo `src`, incluyendo pantallas grandes como `DetallePelicula`, `Dulceria` y `Social`.
- Los flujos mas criticos de usuario se cubren por E2E, pero aun faltan pruebas unitarias especificas para compra de entradas, dulceria y social.
- SonarQube no se puede completar sin servidor accesible y token.

Anexo B. Bitacora completa de ejecucion

Bitacora de pruebas - Filmate UserFrontend

Fecha: 2026-06-23

Entorno: Windows 11, Node.js v24.11.1, npm 11.6.2

Antes

Estado inicial observado:

- No existia script `test`.
- No existia configuracion de cobertura.
- No existia configuracion SonarQube.
- No existian pruebas E2E automatizadas.
- `node_modules` ya estaba presente.
- `npm audit` reporto 3 vulnerabilidades corregibles:
 - 1 baja en `@babel/core`.
 - 1 moderada en `dompurify`.
 - 1 alta en `vite`.

Durante

Cambios aplicados:

- Se instalaron dependencias de prueba:
 - `vitest`
 - `@vitest/coverage-v8`
 - `@testing-library/react`
 - `@testing-library/jest-dom`
 - `@testing-library/user-event`
 - `jsdom`
 - `@playwright/test`
 - `@axe-core/playwright`
 - `sonar-scanner`
- Se ejecuto `npm audit fix`; resultado final: 0 vulnerabilidades.
- Se descargo Chromium de Playwright para pruebas E2E reproducibles.
- Se agregaron scripts npm:
 - `test`
 - `test:watch`
 - `test:coverage`

- `test:e2e`
- `test:e2e:headed`
- `test:all`
- `sonar`
- Se agrego `sonar-project.properties`.
- Se agrego runner E2E propio en `scripts/e2e/run-playwright.mjs` para evitar bloqueos de cierre de servidores en Windows.
- Se agregaron pruebas unitarias/integracion en `src/Component`.
- Se agregaron pruebas E2E en `tests/e2e`.
- Se corrigio un caso real en `normalizeMovie`: ahora los generos/directores/actores anidados tambien son considerados por `parseTextList`.
- Se limpiaron hallazgos de lint:
- imports `React` sin uso.
- valor `allPeliculasSorted` sin uso.
- dependencias de hooks en `MenuPrincipal`.
- callback de login recreado en cada render.
- Se mejoro accesibilidad del login:
- contenedor `main`.
- `h1` accesible visualmente oculto.
- labels asociados a inputs.
- enlace de registro sin boton anidado.
- Se agrego `aria-label/aria-expanded` al boton de menu movil.

Despues

Resultados finales ejecutados:

Comando	Resultado	Notas
<code>npm run lint</code>	OK	0 errores, 0 warnings
<code>npm run test:coverage</code>	OK	4 archivos, 17 tests pasan
<code>npm run test:e2e</code>	OK	8 tests pasan en <code>chromium</code> y <code>mobile-chrome</code>
<code>npm run build</code>	OK	Build generado en <code>dist/</code>
<code>npm audit --json</code>	OK	0 vulnerabilidades
<code>npm run sonar -- --define sonar.host.url=http://127.0.0.1:9000</code>	No ejecutable en este entorno	servidor SonarQube local no disponible, conexion rechazada

Cobertura generada por Vitest:

Metrica	Resultado
Statements	8.54%
Branches	12.65%
Functions	8.47%
Lines	8.94%

Nota: la cobertura global es baja porque se mide todo **src**. Las pruebas actuales priorizan normalizadores, sesion, rutas protegidas, login y flujos E2E. Se recomienda ampliar pruebas unitarias para **DetallePelicula**, **Dulceria** y **Social**.

Advertencias no bloqueantes:

- **npm run build** advierte que un chunk supera 500 kB despues de minificacion. La causa probable es el uso de librerias pesadas como **jspdf** y **html2canvas**. El build termina correctamente.
- SonarQube requiere servidor y token. La configuracion y el reporte **coverage/lcov.info** quedaron listos.

Anexo C. Guia completa de SonarQube

Guia SonarQube - Filmate UserFrontend

Archivos relevantes

- `sonar-project.properties`
- `coverage/lcov.info`
- `package.json`

Preparar cobertura

Ejecutar antes del analisis:

```
npm run test:coverage
```

Esto genera:

```
coverage/lcov.info
```

SonarQube usa ese archivo mediante:

```
sonar.javascript.lcov.reportPaths=coverage/lcov.info
```

Ejecutar contra SonarQube local

Con un servidor SonarQube disponible, por ejemplo `http://127.0.0.1:9000`, ejecutar:

```
npm run sonar -- --define sonar.host.url=http://127.0.0.1:9000 --define sonar.token=<TOKEN>
```

Si el servidor usa una version antigua que no acepta `sonar.token`, usar:

```
npm run sonar -- --define sonar.host.url=http://127.0.0.1:9000 --define sonar.login=<TOKEN>
```

Resultado del intento actual

Se intento ejecutar:

```
npm run sonar -- --define sonar.host.url=http://127.0.0.1:9000
```

Resultado:

```
SonarQube server [http://127.0.0.1:9000] can not be reached  
Connection refused
```

Interpretacion: la configuracion del proyecto y el scanner estan instalados, pero no hay servidor SonarQube escuchando en `127.0.0.1:9000` dentro de este entorno.

Recomendacion CI

Orden sugerido:

```
npm ci
npm run lint
npm run test:coverage
npm run build
npm run test:e2e
npm run sonar -- --define sonar.host.url=$SONAR_HOST_URL --define sonar.token=$SONAR_TOKEN
```

Variables requeridas:

- `SONAR_HOST_URL`
- `SONAR_TOKEN`

No guardar tokens dentro del repositorio.

FILMATE UserFrontend - Informe ampliado generado automaticamente desde scripts/testing/generate-testing-report.mjs